Introduction to Neural Networks

Application to classification

Juan Irving Vásquez
jivg.org

Centro de Innovación y Desarrollo Tecnológico en Cómputo
Instituto Politécnico Nacional

October 12, 2026©



Contents

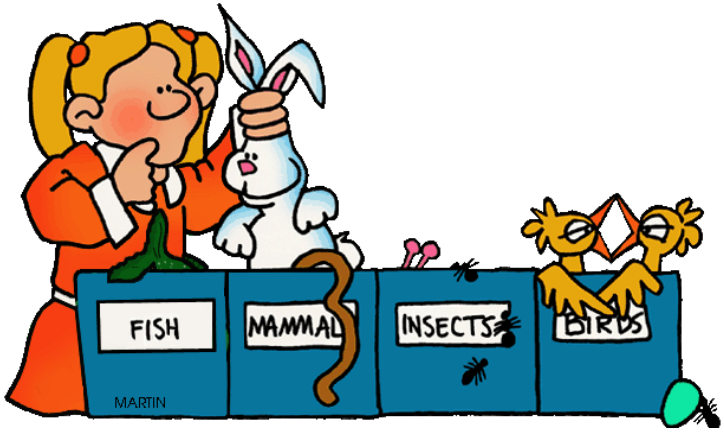
1. Classification
2. Multinomial logistic classifier
3. Data preprocessing
4. Training

Roadmap

1. Classification
2. Multinomial logistic classifier
3. Data preprocessing
4. Training

Classification

- It consist on assigning classes or labels to objects [Sucar 15].



Classification (2)

Unsupervised In this case the classes are unknown. The problem consists in dividing a set of objects into n groups or clusters, so that a class is assigned to each different group. Clustering.

Supervised The possible classes or labels are known a priori, and the problem consists in finding a function or rule that assigns each object to one of the classes.

Target task

Feed forward networks (multilayer perceptrons) can only deal with supervised learning

Supervised Classification

Dataset



New example



Logistic classifier

- Implemented by a neural network with multiple outputs (each output for each class).

$$\hat{Y} = f(XW + B) \tag{1}$$

- lets suppose a new "letter" image comes in.



Logistic classifier

- Implemented by a neural network with multiple outputs (each output for each class).

$$\hat{Y} = f(XW + B) \tag{1}$$

- lets suppose a new "letter" image comes in.



- The output could be:

$$\hat{Y} = f(XW + B) = \begin{bmatrix} 0.9 \\ 1.9 \\ 0.2 \end{bmatrix}$$

- Such outputs ($\hat{y}$) are called logits

Softmax function

Distribution over classes

- We need to convert them into probabilities $p(x = c_i|z)$

$$\hat{Y} = \begin{bmatrix} 0.9 \\ 1.9 \\ 0.2 \end{bmatrix} \rightarrow \begin{bmatrix} p = 0.2 \\ p = 0.7 \\ p = 0.1 \end{bmatrix}$$

- Softmax converts a vector into "probabilities"

$$S(\hat{y}_i) = \frac{e^{\hat{y}_i}}{\sum_j e^{\hat{y}_j}} \tag{2}$$

Softmax function

Distribution over classes

- We need to convert them into probabilities $p(x = c_i|z)$

$$\hat{Y} = \begin{bmatrix} 0.9 \\ 1.9 \\ 0.2 \end{bmatrix} \rightarrow \begin{bmatrix} p = 0.2 \\ p = 0.7 \\ p = 0.1 \end{bmatrix}$$

- Softmax converts a vector into "probabilities"

$$S(\hat{y}_i) = \frac{e^{\hat{y}_i}}{\sum_j e^{\hat{y}_j}} \tag{2}$$

- Then:

$$S(\hat{Y}) = \begin{bmatrix} 0.9 \\ 1.9 \\ 0.2 \end{bmatrix} \rightarrow \begin{bmatrix} p = 0.2 \\ p = 0.7 \\ p = 0.1 \end{bmatrix}$$

The predicted class

- The predicted class is the index with the highest probability:

$$Max(S) = \arg\max_i s_i | s_i \in S \tag{3}$$

[2] Supposing an indexing by one

The predicted class

- The predicted class is the index with the highest probability:

$$Max(S) = \arg\max_i s_i | s_i \in S \tag{3}$$

- Example[2]:

$$S = \begin{bmatrix} p = 0.2 \\ p = 0.7 \\ p = 0.1 \end{bmatrix}$$

$$Max(S) = 2$$

Recall that the second class is 'B'

[2] Supposing an indexing by one

Roadmap

1. Classification
2. Multinomial logistic classifier
3. Data preprocessing
4. Training

Multinomial Logistic Classification

Steps:

1. Given an input X, obtain the neural network outputs:

$$\hat{Y} = f(XW + B)$$

where $\hat{Y}$ is the prediction vector

2. Convert it into probabilities:

$$S(\hat{y}_i) = \frac{e^{\hat{y}_i}}{\sum_j e^{\hat{y}_j}}$$

3. Return the index with highest probability:

$$Max(S) = \arg\max_i s_i | s_i \in S$$

Multinomial Logistic Classification

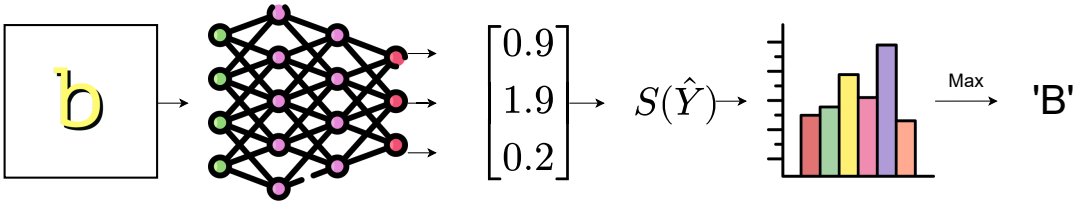


Figure: Classification process

Roadmap

1. Classification
2. Multinomial logistic classifier
3. Data preprocessing
4. Training

Motivation

- The scale and distribution of variables affects stability and speed of gradient descent.
- Poorly scaled data can saturate activations (e.g., sigmoid) and slow down learning.
- Adequate data prevents unbalanced gradients.

Gradient descent:

$$\mathbf{W}^{(t+1)} = \mathbf{W}^{(t)} - \eta \nabla_{\mathbf{W}} L\big(\mathbf{W}^{(t)}\big)$$

The magnitude of ∇L depends on the scale of inputs; rescaling/standardizing improves numerical conditioning.

Data transforms: overview

- Creating different "views" of the dataset can better reveal its structure.
- Two typical phases: *fit* the parameters of the transform and *apply* it.
- In neural networks, transformations support more stable optimization.
- Formally,

$$\mathbf{x}' = T(\mathbf{x}; \boldsymbol{\theta}), \tag{4}$$

$$\boldsymbol{\theta} \leftarrow \text{fit}(\mathcal{D})$$

Parameters $\boldsymbol{\theta}$ are estimated on training data, then T is applied to train/validation/test sets.

Rescaling (Min–Max)

- Homogenizes ranges (e.g., to $[0, 1]$ or $[a, b]$).
- Useful for activation functions sensitive to magnitude and for training stability.
- Given a range $[a, b]$:

$$T_{\text{minmax}}(x_i; (a, b, X)) \;=\; a \;+\; \frac{x_i - \min(X)}{\max(X) - \min(X)}\,(b - a) \tag{5}$$

Common case: $a = 0$, $b = 1$. Fit only on training data to avoid data leakage.

Rescaling (Min-max)

Example

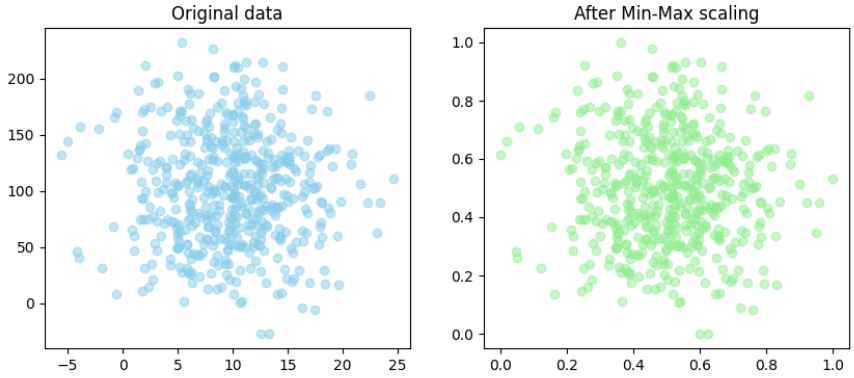


Figure: Rescaling keeps the distribution

Standardization (Z–score)

- Centers and scales each variable to mean 0 and standard deviation 1.
- Balances weight updates and ensures well-conditioned gradients.

$$T_{\text{std}}(x_i; (\mu_i, \sigma_i)) = \frac{x_i - \mu_i}{\sigma_i} \tag{6}$$

$$\text{with} \quad \mu_i = \mathbb{E}[X], \quad \sigma_i = \sqrt{\mathbb{V}[X]}$$

Estimate μ_i, σ_i on training data and reuse them for validation/test.

Standardization

Example

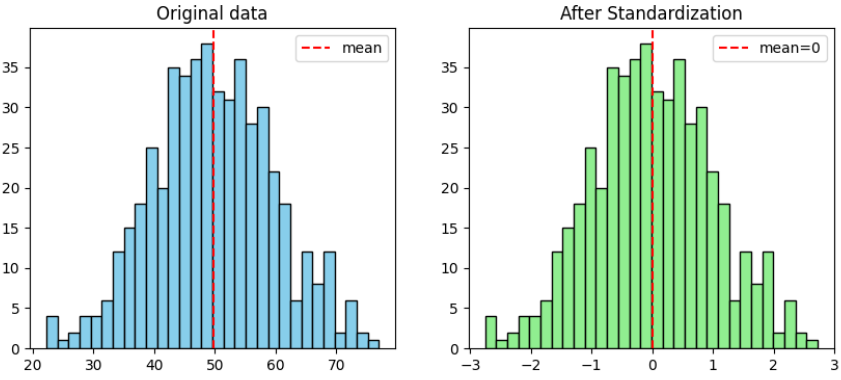


Figure: Data is adjusted to fit a zero centered distribution.

Normalization (unit norm)

- Rescales each *row* to have norm 1 (typically L2).
- Useful when the direction of the vector matters more than its magnitude.
- Formally,

$$T_{\text{norm}}(\mathbf{x}) = \frac{\mathbf{x}}{\|\mathbf{x}\|_2} \tag{7}$$

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{j=1}^{d} x_j^2} \tag{8}$$

- Other norms (e.g., L1) can also be used depending on the problem.

Normalization

Example

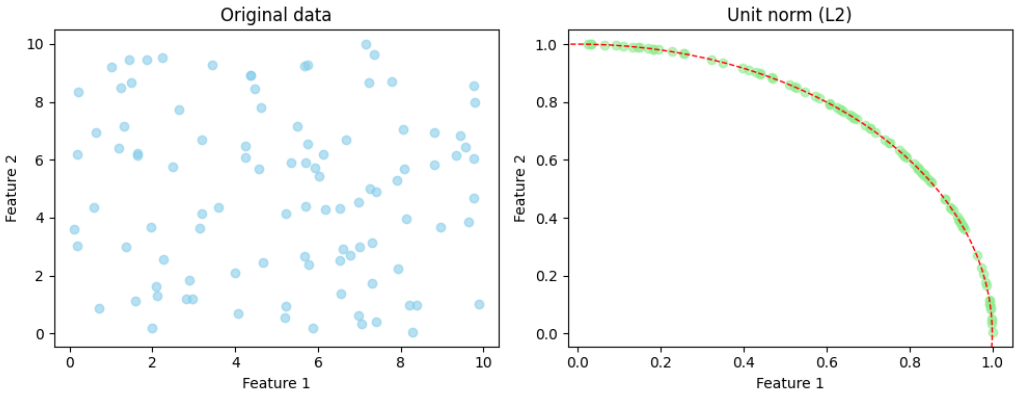


Figure: In normalization the angles and differences are important

Binarization (threshold)

- Converts continuous variables into binary ones based on a threshold τ.
- Useful for feature engineering or discrete decision-making.
- Formally,

$$T_{\text{bin}}(x) = \mathbf{1}\{x > u\} = \begin{cases} 1, & x > u \\ 0, & x \leq u \end{cases} \tag{9}$$

Threshold u is chosen by domain knowledge or validation.

Binarization

Example



Figure: Binarization of an image.

One-hot encoding

Target labels preprocessing

- Facilitates the representation of categories in neural networks
- They are used to train the network
- Lets suppose the input image:



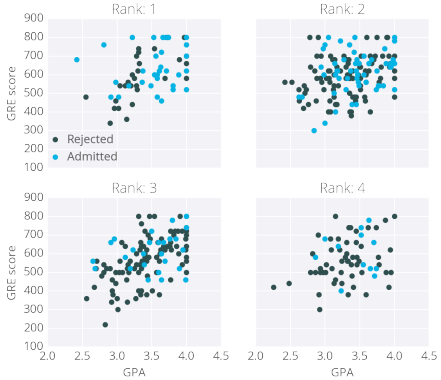
- Then its label 'A' is converted

$$L('A') = \begin{bmatrix} 1.0 \\ 0.0 \\ 0.0 \end{bmatrix}$$

Categorical Data

One-hot encoding for features example

- Admissions (again..). This dataset has three input features: GRE score, GPA, and the rank of the undergraduate school (numbered 1 through 4). Institutions with rank 1 have the highest prestige, those with rank 4 have the lowest.



Categorical Data

One-hot encoding for features

- Numerical and categorical data
- Add dummy variables.

	admit	gre	gpa	rank_1	rank_2	rank_3	rank_4
15	0	-0.932334	0.131646	0	0	1	0
115	0	0.279614	1.576859	0	0	1	0
55	1	1.318426	1.603135	0	0	1	0
175	1	0.279614	-0.052290	0	1	0	0
63	1	0.799020	1.208986	0	0	1	0
67	0	0.279614	-0.236227	1	0	0	0
216	0	-2.144282	-1.287291	1	0	0	0
145	0	-1.798011	0.105369	0	0	1	0
286	1	1.837832	-0.446439	1	0	0	0
339	1	0.625884	0.210476	0	0	1	0

Roadmap

1. Classification
2. Multinomial logistic classifier
3. Data preprocessing
4. Training

Multinomial Logistic Classification (2)

- Multinomial Logistic classifier

$$S(f(XW + B))$$

Multinomial Logistic Classification (2)

- Multinomial Logistic classifier

$$S(f(XW + B))$$

- Do we hit?



- To get the loss or error we measure the distance with respect to one-hot target vectors

$$D(S, L)$$

Cross Entropy

Loss function

- Measures the distance between two probability vectors:

$$D(S, L) = -\sum_i l_i \log(s_i) \tag{10}$$

where L is the target label and S the softmax.

- For N examples:

$$D(S, L) = -\frac{1}{N} \sum_\mu \sum_i l_i^\mu \log(s_i^\mu) \tag{11}$$

- Substitution

$$D(S(f(XW + B)), L)$$

- $D(S, L) \neq D(L, S)$

Binary Cross Entropy

Loss function

- Measures the distance

$$L(y, p) = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(p_i) + (1 - y_i)\log(1 - p_i)] \tag{12}$$

  where L is the target label.
- $L(y, p)$ is the binary cross entropy loss.
- N is the number of samples.
- y_i is the actual label of the ith sample
- p_i is the predicted probability of the sample being in the positive class.

Training the Neural Network

- Multinomial Logistic classifier

$$D(S(f(XW + B)), L)$$

Training the Neural Network

- Multinomial Logistic classifier

$$D(S(f(XW + B)), L)$$

- What do we need to do?

Training the Neural Network

- Multinomial Logistic classifier

$$D(S(f(XW + B)), L)$$

- What do we need to do?
- Decrease the distances

$$\downarrow D(A, a)$$

$$\uparrow D(A, \neg a)$$

Training the Neural Network

- Multinomial Logistic classifier

$$D(S(f(XW + B)), L)$$

- What do we need to do?
- Decrease the distances

$$\downarrow D(A, a)$$
$$\uparrow D(A, \neg a)$$

- Loss = average cross entropy

$$\mathcal{L} = \frac{1}{N} \sum_i D(S(f(Wx_i + B)), L_i) \tag{13}$$

Minimizing the cross entropy



Figure: Training with gradient descent

Minimizing the cross entropy

- The weights are updated with: Δw

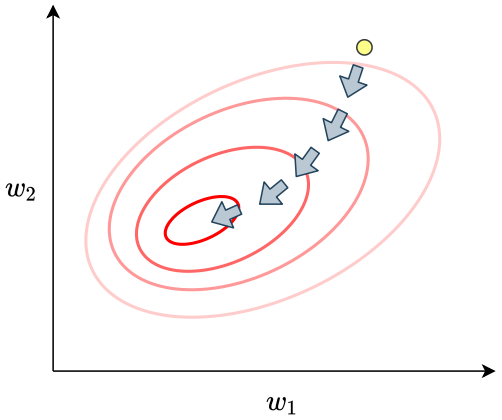


Figure: Gradient descent for $\mathcal{L}(w_1, w_2)$.

Training workflow

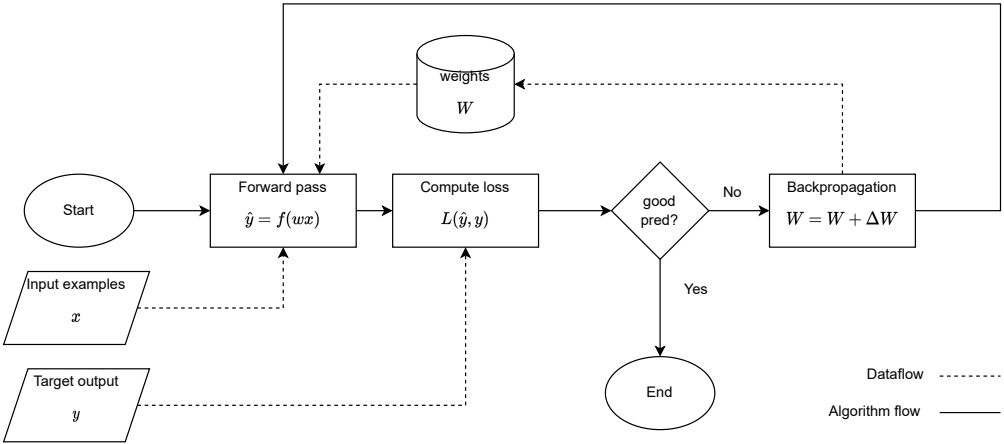
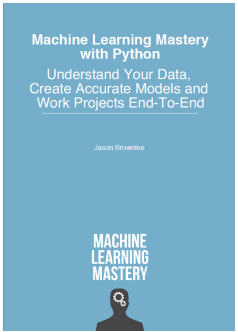


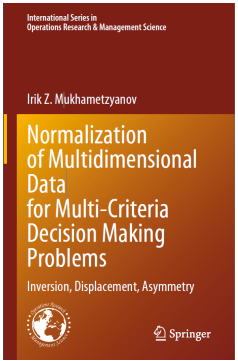
Figure: Neural network training by gradient descent

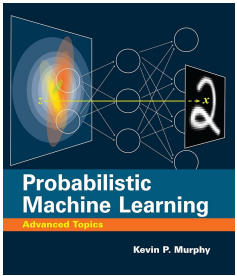
Lectures

- Jorge Luis Borges. Funes el memorioso.

References


Machine Learning Mastery with Python
Understand Your Data, Create Accurate Models and Work Projects End-To-End
Jason Brownlee
MACHINE LEARNING MASTERY


International Series in Operations Research & Management Science
Irik Z. Mukhametzyanov
Normalization of Multidimensional Data for Multi-Criteria Decision Making Problems
Inversion, Displacement, Asymmetry
Springer


Probabilistic Machine Learning
Advanced Topics
Kevin P. Murphy